

EXPERT TIPS

Integer positions

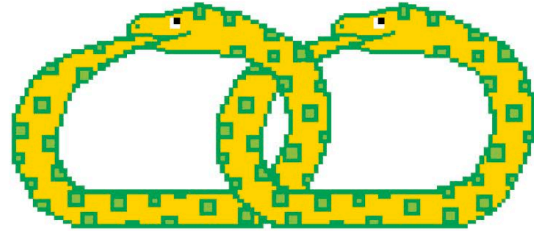
You use the value `len(message) / 2` in your loop range because the even and odd letter lists are both half the length of the original message. You made sure the length of your message will always be even by getting the program to add an x when necessary, so it can be divided by 2. However, the result will be a float value (with a decimal point, such as 3.0 or 4.0) rather than an integer (a whole number, such as 3 or 4). Python gives an error if you try to use a float for the position of an item in a list, so use the `int()` function to convert it to an integer.

```
>>> mystring = 'secret'
>>> mystring[3.0]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    mystring[3.0]
TypeError: string indices must be integers
```

This is the error message Python will give you if you use a float, such as 3.0, instead of an integer, such as 3.

12 Update the loop

The `swap_letters()` function has a really useful feature: if you run it on an encrypted message, it will decrypt it. So you can use this function to encrypt or decrypt messages depending on what the user wants to do. Make the following changes to the `while` loop you created in Step 6.



```
while True:
```

```
    task = get_task()
```

```
    if task == 'encrypt':
```

```
        message = get_message()
```

```
        encrypted = swap_letters(message)
```

```
        messagebox.showinfo('Ciphertext of the secret message is:', encrypted)
```

```
    elif task == 'decrypt':
```

```
        message = get_message()
```

```
        decrypted = swap_letters(message)
```

```
        messagebox.showinfo('Plaintext of the secret message is:', decrypted)
```

```
    else:
```

```
        break
```

```
root.mainloop()
```

Use `swap_letters()` to encrypt the message.

Display the encrypted message.

Uses `swap_letters()` to decrypt the message.

Display the decrypted message.